

1.版本历史

日期	版本号	作者	标注
2020.11.16	V1.0.0		初定协议内容
2020.11.19	V1.1.0		完善协议内容，新增控制类型和字段
2020.11.20	V1.2.0		新增座位，车镜控制
2020.11.23	V1.3.0		新增文档描述
2020.12.02	V1.4.0		新增 Carlife车控 交互流程，字段增加，DEMO 例子说明
2020.12.14	V1.4.1		定义serviceType
2020.12.15	V1.4.2		修改proto值

2.文档介绍

1. 此文档为 Carlife+车控协议的初定版本，仅供参考。后续会更新上 Carlife+的Spec 文档，最终以 Spec文档作为集成标准。
2. Carlife+车控协议，复用CMD 通道来传输，请参考 Spec 文档 - 《10.通信协议终；10.2.控制消息》。
3. **MSG_CMD_VEHICLE_CONTROL** (MD -> HU) ， service type：0x0001006F。移动设备发送给车机。根据 type 可以判定是 get （获取车辆信息，车机需要回复MSG_CMD_VEHICLE_INFO）还是 post （车控指令）
4. **MSG_CMD_VEHICLE_CONTROL_INFO** (HU -> MD) ， service type：0x00018061。车机发送给移动设备的车机控制信息。当车辆各个模块状态有变动可以主动推送，若收到**MSG_CMD_VEHICLE_CONTROL** 的 type为 get 的时候也需要把当前状态发送给移动设备。

3.车控协议结构

1.MSG_CMD_VEHICLE_CONTROL - Proto结构

```
message CarlifeVehicleControl {  
    // 指令类型  
    required int32 type = 1;  
    // 属性id  
    required int32 id = 2;  
    // 预留字段  
    optional boolean support = 3;  
    // 预留字段
```

```

optional string token_string = 4;
// 车控区域id
optional int32 area_id = 5;
// 车控具体区域集合
repeated int32 area_value = 6;
// 属性值的类型
optional int32 value_type = 7;
// byte类型的属性值
optional bytes bytes_value = 8;
// int32类型的属性值
repeated sint32 int32_values = 9;
// int64类型的属性值
repeated sint64 int64_values = 10;
// float类型的属性值
repeated float float_values = 11;
// string类型的属性值
optional string string_value = 12;
}

```

2.MSG_CMD_VEHICLE_CONTROL_INFO - Proto结构

```

message CarlifeVehicleDataList {
    // 是否支持车控
    required int32 support = 1;
    // 预留字段
    optional string token_string = 2;
    // 汽车信息集合
    repeated CarlifeVehicleData vehicleInfo = 3;
}

message CarlifeVehicleData {
    // 信息类型
    required int32 id = 1;
    // 是否支持
    optional bool support = 2;
    // 区域ID, 中控/车窗/车座等
    optional int32 area_id = 3;
    // 区域列表, 前车窗/后车窗, 主驾驶位座位/副驾驶座位等
    repeated int32 area_value = 4;
    // 信息值的类型
    optional int32 value_type = 5;
    // byte数组属性值
    optional bytes bytes_value = 6;
    // int32类型的属性值
    repeated sint32 int32_values = 7;
    // int64类型的属性值

```

```
repeated sint64 int64_values = 8;
// float类型的属性值
repeated float float_values = 9;
// string类型的属性值
optional string string_value = 10;
}
```

4.车控交互的枚举类型说明

1.MSG_CMD_VEHICLE_CONTROL

1.1type

```
// 获取车控信息
GET = 0x0001;
// 发起车控指令
POST = 0x0002;
```

1.2.id

MOVE 与 POS 的关系介绍：

1.MOVE 行为可以是一个持续性的过程，并且正值代表正向移动，负值是反向移动。例如，WINDOW_MOVE，如果携带参数1，代表车窗车窗向下移动，如果携带参数-1，代表车窗向上移动。官方解释：Positive value indicates tilt upwards, negative value is downwards

2.POS 行为是一次性行为的过程，如果携带参数为最大值（由车机决定最大值为多少），则代表打开，如果携带参数为0，那么代表关闭。例如，WINDOW_POS，如果携带参数为最大值的一半，代表车窗移动到一半。官方解释：Max value indicates fully open, min value (0) indicates fully closed

注意，type 为 POST的情况下的 ID 值：

```
// 控制车窗移动
WINDOW_MOVE = 0x0BC1,
// 控制车窗到具体位置
WINDOW_POS = 0x0BC0,
// 控制空调开/关
HVAC_POWER_ON = 0x0510,
// 控制空调温度
HVAC_TEMPERATURE_SET = 0x0503,
// 控制空调风速
HVAC_WIND_SET = 0x0504,
// 控制空调风速类型（手动/自动）
HVAC_WIND_TYPE_SET = 0x0505,
```

```

// 控制车座向前向后移动
SEAT_FORE_AFT_MOVE = 0x0B86,
// 控制车座靠背向前向后移动
SEAT_BACKREST_ANGLE_MOVE = 0x0B88,
// 控制车镜开/关
MIRROR_ON = 0x0B46,
// 控制车镜y轴到具体位置
MIRROR_Y_POS = 0x0B42,
// 控制车镜y轴移动
MIRROR_Y_MOVE = 0x0B43,
// 控制车镜z轴到具体位置
MIRROR_Z_POS = 0x0B40,
// 控制车镜z轴移动
MIRROR_Z_MOVE = 0x0B41,

```

注意, type 为 GET 的情况下的 ID 值:

```

// 车辆所有信息
GET_ALL_STATUS = 0x0001;
// 车辆基础信息
GET_BASE_STATUS = 0x0002;
// 空调信息
GET_HVAC_STATUS = 0x0003;
// 车镜信息
GET_MIRROR_STATUS = 0x0004;
// 座位信息
GET_SEAT_STATUS = 0x0005;
// 车窗信息
GET_WINDOW_STATUS = 0x0006;

```

1.3.value_type

```

/**
 * Enumerates supported data types for VehicleProperty.
 *
 * This is a bitwise flag that supposed to be used in VehicleProperty enum.
 */
enum VehiclePropertyType : int32_t {
    STRING          = 0x00100000,
    BOOLEAN         = 0x00200000,
    INT32           = 0x00400000,
    INT64           = 0x00500000,
    FLOAT           = 0x00600000,
    BYTES           = 0x00700000,
};

```

1.4.area_id

```
/**
 * Some properties may be associated with particular vehicle areas. For
 * example, VehicleProperty:DOOR_LOCK property must be associated with
 * particular door, thus this property must be marked with
 * VehicleArea:DOOR flag.
 *
 * Other properties may not be associated with particular vehicle area,
 * these kind of properties must have VehicleArea:GLOBAL flag.
 *
 * This is a bitwise flag that supposed to be used in VehicleProperty enum.
 */
enum VehicleArea : int32_t {
    // 如果没有具体区域, 或者代表具体数值, 则携带 GLOBAL
    GLOBAL          = 0x01000000,
    // 代表中控台
    ZONE            = 0x02000000,
    // 代表车窗
    WINDOW          = 0x03000000,
    // 代表车镜
    MIRROR          = 0x04000000,
    // 代表车座
    SEAT            = 0x05000000,
    // 代表车门
    DOOR            = 0x06000000,
};
```

1.5.area_value

```
/**
 * Various windshields/windows in the car.
 * 车窗具体位置枚举
 */
enum VehicleAreaWindow : int32_t {
    // 前车窗
    FRONT_WINDSHIELD = 0x0001,
    // 后车窗
    REAR_WINDSHIELD = 0x0002,
    // 天窗
    ROOF_TOP = 0x0004,
    // 主驾驶位车窗
    ROW_1_LEFT = 0x0010,
```

```

    // 副驾驶位车窗
    ROW_1_RIGHT = 0x0020,
    // 第二排左边车窗
    ROW_2_LEFT = 0x0100,
    // 第二排右边车窗
    ROW_2_RIGHT = 0x0200,
    // 第三排左边车窗
    ROW_3_LEFT = 0x1000,
    // 第三排右边车窗
    ROW_3_RIGHT = 0x2000,
    // 全部车窗
    ALL = 0x3000,
};

/**
 * Various Seats in the car.
 * 车座位具体位置枚举
 */
enum VehicleAreaSeat : int32_t {
    // 主驾驶位座位
    ROW_1_LEFT = 0x0001,
    // 第一排中间座位（未来可能会有）
    ROW_1_CENTER = 0x0002,
    // 副驾驶位座位
    ROW_1_RIGHT = 0x0004,
    // 第二排左边座位
    ROW_2_LEFT = 0x0010,
    // 第二排中间座位
    ROW_2_CENTER = 0x0020,
    // 第二排右边座位
    ROW_2_RIGHT = 0x0040,
    // 第三排左边座位
    ROW_3_LEFT = 0x0100,
    // 第三排中间座位
    ROW_3_CENTER = 0x0200,
    // 第三排右边座位
    ROW_3_RIGHT = 0x0400
};

/**
 * Various Mirror in the car.
 * 车镜位具体位置枚举
 */
enum VehicleAreaMirror : int32_t {
    // 车外左边车镜
    DRIVER_LEFT = 0x00000001,
    // 车内后视镜
    DRIVER_RIGHT = 0x00000002,
    // 车外右边车镜

```

```
DRIVER_CENTER = 0x00000004,  
};
```

2.MSG_CMD_VEHICLE_CONTROL_INFO

2.1.support

```
enum VehicleControlInfoSupport : int32_t {  
    // 支持车控  
    FUNCTION_ENABLE = 0x0001;  
    // 不支持车控  
    FUNCTION_DISABLE = 0x0002;  
};
```

2.2.CarlifeVehicleData

2.2.1.id

```
-- 车辆基础信息 (BASE)  
// 汽车是否启动  
VEHICLE_BASE_POWER_STATUS = 0x0001;  
// 汽车是否在行驶  
VEHICLE_BASE_DRIVE_STATUS = 0x0002;  
// 汽车行驶速度单位 (公里/时、英里/时)  
VEHICLE_BASE_SPEED_TYPE = 0x0003;  
// 汽车行驶速度 (float)  
VEHICLE_BASE_SPEED_VALUE = 0x0004;  
  
-- 空调信息 (HVAC)  
// 空调是否打开  
HVAC_POWER_STATUS = 0x0005;  
// 空调温度单位 (华氏度/摄氏度)  
HVAC_TEMPERATURE_TYPE = 0x0006;  
// 空调目前温度值 (float)  
HVAC_TEMPERATURE_VALUE = 0x0007;  
// 空调风速类型 (自动/手动)  
HVAC_WIND_TYPE = 0x0008;  
// 空调风速最高档位 (int32)  
HVAC_WIND_MAX_LEVEL = 0x0009;  
// 空调目前风速档位 (int32)  
HVAC_WIND_SPEED = 0x000A;  
  
-- 车镜信息 (MIRROR)  
// 车镜是否打开
```

```
MIRROR_STATUS = 0x000B;

// 持续更新
...
```

2.2.2.value_type

```
enum CarlifeVehicleInfoValueType : int32_t {
    STRING          = 0x00100000,
    INT32           = 0x00200000,
    INT64           = 0x00300000,
    FLOAT           = 0x00400000,
};
```

2.2.3.value

```
// 开或关通用枚举
enum CarlifeVehicleInfoStatus : int32_t {
    ON = 0x0001;
    OFF = 0x0002;
};

// 速度单位（公里/时、英里/时）
enum CarlifeVehicleInfoBaseSpeedType : int32_t {
    // 公里/时
    KMH = 0x0001;
    // 英里/时
    MPH = 0x0002;
};

// 空调温度单位（摄氏度/华氏度）
enum CarlifeVehicleInfoHvacTemType : int32_t {
    // 摄氏度
    CENTIGRADE = 0x0001;
    // 华氏度
    FAHRENHEIT = 0x0002;
};

// 空调风速类型（自动/手动）
enum CarlifeVehicleInfoHvacWindType : int32_t {
    // 自动
    AUTO = 0x0001;
    // 手动
    MANUAL = 0x0002;
};
```

5.车辆控制细节

1.车窗控制

1.1.说明

由 Carilfe+发送到车机，进行车窗的控制。

1.2.例子

主驾驶位 车窗降低 (value : 1代表升高, -1代表降低)

```
id = WINDOW_MOVE(0x0BC1)
value_type = INT32(0x00400000)
area_id = WINDOW(0x03000000)
area_value = [VehicleAreaWindow_ROW_1_LEFT(0x0010)]
int32_values = [-1]
```

全部车窗升高 (value : 1代表升高, -1代表降低)

```
id = WINDOW_MOVE(0x0BC1)
value_type = INT32(0x00400000)
area_id = WINDOW(0x03000000)
area_value = [VehicleAreaWindow_ALL]
int32_values = [1]
```

主驾驶位车窗 打开

```
id = WINDOW_POS(0x0BC0)
value_type = INT32(0x00400000)
area_id = WINDOW(0x03000000)
area_value = [VehicleAreaWindow_ROW_1_LEFT(0x0010)]
int32_values = [Integer.MAX_VALUE(2147483647)]
```

2.空调控制

1.1说明

由 Carilfe+发送到车机，进行空调的控制。

1.2.例子

打开空调 (value : 1代表开, -1代表关)

```
id = HVAC_POWER_ON(0x0510)
```

```
value_type = INT32(0x00400000)
```

```
area_id = ZONE(0x02000000)
```

```
int32_values = [1]
```

设置空调温度为26度

```
id = HVAC_TEMPERATURE_SET(0x0503)
```

```
value_type = FLOAT(0x00600000)
```

```
area_id = ZONE(0x02000000)
```

```
float_values = [26.0]
```

空调风速提高一档 (value : 1代表升高, -1代表降低)

```
id = HVAC_WIND_SET(0x0504)
```

```
value_type = INT32(0x00400000)
```

```
area_id = ZONE(0x02000000)
```

```
int32_values = [1]
```

空调风速降低一档 (value : 1代表升高, -1代表降低)

```
id = HVAC_WIND_SET(0x0504)
```

```
value_type = INT32(0x00400000)
```

```
area_id = ZONE(0x02000000)
```

```
int32_values = [-1]
```

空调风速切换为自动模式 (value : 1代表自动, -1代表手动)

```
id = HVAC_WIND_TYPE_SET(0x0505)
```

```
value_type = INT32(0x00400000)
```

```
area_id = ZONE(0x02000000)
```

```
int32_values = [1]
```

3.车座控制

1.1说明

由 Carilfe+发送到车机，进行车辆座位的控制。

1.2.例子

主驾驶位 座位 往后调 (value : 1代表向前, -1代表向后)

```
id = SEAT_FORE_AFT_MOVE(0x0B86)
```

```
value_type = INT32(0x00400000)
```

```
area_id = SEAT(0x05000000)
```

```
area_value = [VehicleAreaSeat_ROW_1_LEFT(0x0001)]
```

```
int32_values = [-1]
```

副驾驶位 座位 往前调 (value : 1代表向前, -1代表向后)

```
id = SEAT_FORE_AFT_MOVE(0x0B86)
```

```
value_type = INT32(0x00400000)
```

```
area_id = SEAT(0x05000000)
```

```
area_value = [VehicleAreaSeat_ROW_1_RIGHT(0x0004)]
```

```
int32_values = [1]
```

副驾驶位 座位靠背 往后调 (value : 1代表向前, -1代表向后)

```
id = SEAT_BACKREST_ANGLE_MOVE(0x0B88)
```

```
value_type = INT32(0x00400000)
```

```
area_id = SEAT(0x05000000)
```

```
area_value = [VehicleAreaSeat_ROW_1_RIGHT(0x0004)]
```

```
int32_values = [-1]
```

4.车镜控制

1.1说明

由 Carilfe+发送到车机，进行车辆座位的控制。

1.2.例子

两边车镜打开 (value : 1代表开, -1代表关)

```
id = MIRROR_ON(0x0B46)
```

```
value_type = INT32(0x00400000)
```

```
area_id = MIRROR(0x04000000)
```

```
area_value = [DRIVER_LEFT(0x00000001),DRIVER_RIGHT(0x00000002)]
```

```
int32_values = [1]
```

副驾驶位 车镜负Y轴调整 (value : 1代表正 Y 轴移动, -1代表负 Y 轴移动)

```
id = MIRROR_Y_MOVE(0x0B43)
```

```
value_type = INT32(0x00400000)
```

```
area_id = MIRROR(0x04000000)
```

```
area_value = [DRIVER_RIGHT(0x00000002)]
```

```
int32_values = [-1]
```

主驾驶位 车镜正Z轴拉满

```
id = MIRROR_Z_POS(0x0B40)
```

```
value_type = INT32(0x00400000)
```

```
area_id = MIRROR(0x04000000)
```

```
area_value = [DRIVER_LEFT(0x00000001)]
```

```
int32_values = [Integer.MAX_VALUE(2147483647)]
```

5.后尾箱 + 前车盖 + 油箱盖 控制

暂无

